Lab 2

Processes and channels

SENTINEL Lab 2: libproc. One process for each sensor, and one pipe from each sensor to the centre.

Where Lab 2 fits

Lab 1 gave the centre a feed, a clock and a mission log. Lab 2 starts the four sensors as processes, and brings their reports to the centre.

Lab 1 · Oct. 9

libsyscall

The interface to the operating system: the feed, the mission log and precise timestamps.

R6

Read a feed and lose none of it. Put a real date on each report. Record each report, in order.

Lab 2 · Oct. 30 · this lab

libproc

One process for each sensor, and the channels that bring their reports to the centre.

R1 R2

Continue when one sensor stops. Let an urgent report overtake routine traffic.

Lab 3 · Nov. 20

libipc

Worker threads that share one processor and keep the track picture.

R3

Take thousands of reports each second. Measure what shared data with no lock costs.

Lab 4 · Dec. 4

libsync

Protection for each thing that the workers share.

R4 R5

Keep the picture exact while many workers change it. Never freeze.

Lab 2 at a glance

Oct. 28

Prelab due by midnight.
Necessary, no marks.

Oct. 30

Lab session. The Lab 1 oral exam is in this session.

Nov. 12

Deliverable due by midnight.
80 marks, 5% of the grade.

Nov. 20

Oral exam in the Lab 3 session.
20 marks, 3% of the grade.

The 80 marks of the deliverable

20	24	12	12	4	8
Acceptance test	Course tests	Your test	Code rules and errors	valgrind, no zombie	Three files agree

What you write in Lab 2

`src/libproc/src/sensor.c`	sensor_spawn, sensor_stop, sensor_reap
`src/libproc/src/prio_channel.c`	prio_open, prio_close, prio_send
`src/libproc/src/sched.c`	sched_create, sched_destroy, sched_push, sched_pending
`src/libproc/tests/test_lab2.c`	Three test cases, with assert()
`src/libsyscall/src/io.c`	Lab 1: add O_CLOEXEC to each open() of the three sc_open_* functions

Read these first. Do not change them

`proc.h`

The steps, the errors and the log lines of each function.

`given/centre.c`

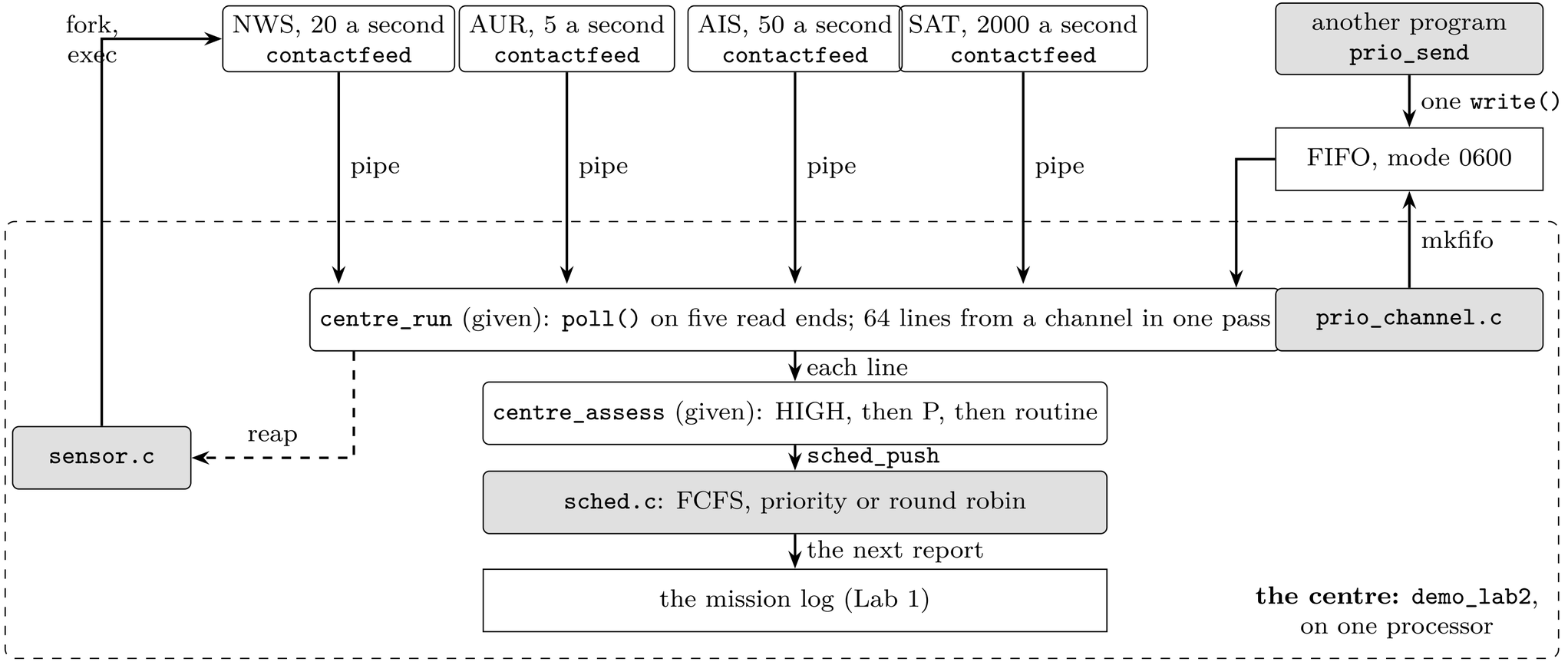
The loop of the centre. It calls each of your functions.

`given/demo_lab2.c`

The demo. Its first comment gives its options.

Each function comes as a stub, with its steps in a comment above it.

How Lab 2 works



Grey: your code. White: given code. Square corners: a file or a FIFO.

The ideas of Lab 2

fork() makes a child process

fork() copies the centre into a new child process; fork() returns 0 in the child and the pid of the child in the centre.

`Brief 3.1`

execvp() runs the sensor program

execvp() replaces the program of the child with contactfeed; each descriptor with O_CLOEXEC closes, so the sensor gets no copy of the mission log.

`Brief 3.2`

Close the pipe end that you do not use

The centre sees that a sensor stopped only when no process holds the write end of its pipe, so the centre closes its own copy.

`Brief 3.3`

waitpid() collects a stopped sensor

A sensor that stopped stays a zombie until the centre calls waitpid(); with WNOHANG, the call returns at once when the sensor still runs.

`Brief 3.4`

A FIFO brings urgent contacts

A FIFO is a pipe with a file name, so any program on the machine can open it and send an urgent contact to the centre.

`Brief 3.5`

The scheduler chooses the next report

When many reports wait, FCFS takes the oldest, priority takes the most urgent, and round robin takes one from each sensor in turn.

`Brief 3.6`

Run it

Run each command at the top of your repository.

`make`	Builds src/libproc. Until your files exist, it names each missing file.
`make lab2 KILL=AUR`	Runs the demo for 3 s with your library, and kills AUR after 0.5 s.
`make test LAB=2`	Runs your test, test_policies, and the tests of Lab 1.
`make accept2`	Runs the 26 checks of the acceptance test.
`make submit LAB=2`	Writes the zip of Lab 2.

Done when, then hand in

- [ ] make prints no warning.
- [ ] make test LAB=2 passes your test and test_policies.
- [ ] make accept2 gives 26 of 26.
- [ ] The record run exits with status 0. Each requirement line says yes.
- [ ] After a run, no contactfeed process runs, and the FIFO does not exist.
- [ ] valgrind reports no error and no leak.
- [ ] Lab 1 is still complete.

Then hand in

```
make submit LAB=2
sentinel-lab2-<name>.zip
```

Put the three files of one record run in docs/lab2/: missions.log, demo_output.txt and proc_capture.txt.

Lab 2 Brief 6 gives the commands.

Upload the zip to Brightspace by Nov. 12, midnight.

Questions?

If you have no questions, start work on Lab 2.

Read these two files first:

`docs/lab2/lab2_brief.pdf`

`src/libproc/include/sentinel/proc.h`